COMPLETE JAVASCRIPT PROGRAMMING SERIES

MODULE 1

INTRODUCTION TO JAVASCRIPT

Authors: Mithila Rani Saha & Dithsa Dutta

IT STUDY HUB

www.itstudyhub.dpdns.org

About This Module

Welcome to Volume 1 of the IT Study Hub Complete JavaScript Programming Series. This 12-volume series is designed to take you from absolute zero programming knowledge to professional frontend developer architecture.

Module 1 acts as your foundation. Before we can build skyscrapers, we must understand the tools, the history, and the ground we are building upon. You will not be building complex applications today. Instead, you will learn *how* to speak to the computer, *where* to write your code, and *why* JavaScript behaves the way it does.

LEARNING OUTCOMES

- Understand the core purpose and history of JavaScript in web development.
- Differentiate JavaScript from Java and understand its modern environments (Node.js).
- Successfully write, execute, and debug basic scripts using the Browser Console.
- Grasp the fundamental syntax rules, expressions, and the importance of Strict Mode.
- Integrate JavaScript into an HTML document.

MODULE COVERAGE

Topic	Description
1. Theory & Context	What is JS, History, JS vs Java, Use cases, JS Engines
2. Environment Setup	Running JS, Browser Console, HTML Integration, Node.js
3. Fundamentals	Syntax, Comments, Statements vs Expressions, Strict Mode
4. Tooling	Developer Tools, Debugging

IT STUDY HUB

What is JavaScript?

1. CONCEPT

> **JavaScript is a dynamic, high-level programming language primarily used to make webpages interactive.**

2. SIMPLE EXPLANATION

Think of a webpage as a car. HTML is the body and frame (the structure). CSS is the paint, seats, and design (the styling). JavaScript is the engine, steering wheel, and pedals (the behavior). Without JavaScript, a webpage is just a static poster. With JavaScript, it becomes a living, breathing application that responds to your clicks, scrolls, and typing. It allows you to update content on the fly, animate images, and control multimedia.

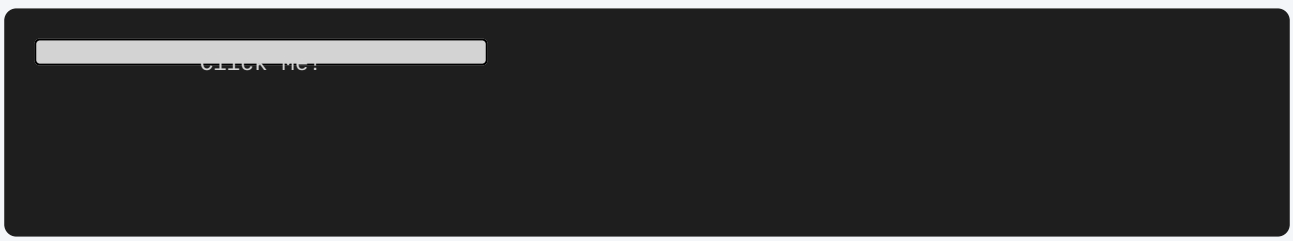
3. SYNTAX

```
// Standard syntax for changing HTML content
document.getElementById('element_id').innerHTML = 'New Content';
```

4. EASY EXAMPLE

Let us imagine we have a simple button on our website. When the user clicks this button, we want a greeting message to appear on the screen.

5. CODE



6. OUTPUT

```
Output:
Before click: [Click Me!] (Empty text below)
After click: [Click Me!]
Hello, Welcome to JavaScript!
```

7. CODE EXPLANATION

Line 1-2: We create a button and an empty paragraph in HTML.
Line 5: We tell JavaScript to find the button using its ID 'greetBtn' and listen for an 'onclick' event.
Line 6: When the click happens, JavaScript finds the empty paragraph ('message') and changes its internal HTML to our welcome text.

8. REAL-WORLD USE

Showing a popup modal when a user tries to leave a page, auto-updating a live sports score without reloading the page, or creating a web-based calculator.

9. COMMON MISTAKES

Caution: Assuming JavaScript is only for the browser (it can be used on servers too), or confusing the ID names between HTML and JavaScript.

10. PRACTICE

Task: Write a line of JavaScript that finds an HTML element with the ID 'title' and changes its text to 'Module 1 Complete'.

History and evolution of JavaScript

1. CONCEPT

> **JavaScript was created in just 10 days in 1995 and has evolved into the most widely used programming language in the world.**

2. SIMPLE EXPLANATION

In the early days of the internet, webpages were completely static (like reading a physical newspaper). Netscape, a popular browser company, wanted to add 'glue' language to make websites interactive. They hired Brendan Eich, who wrote the first version of JavaScript in just 10 days in May 1995. Originally called 'Mocha', then 'LiveScript', it was finally renamed 'JavaScript' as a marketing tactic because Java was very popular at the time. Today, it is governed by an official standard called ECMAScript.

3. SYNTAX

```
// Old way (ES5 - 2009)
var name = 'John';

// Modern way (ES6 - 2015 onwards)
let name = 'John';
const pi = 3.14;
```

4. EASY EXAMPLE

Over time, the language added new features to make writing code easier and safer. Let's look at how variable declaration evolved.

5. CODE

```
// 1995 style: No strict rules, confusing behavior
name = "Brendan";

// 2009 style (ES5): Slightly better but 'var' has flaws
var language = "JavaScript";

// 2015 style (ES6): The modern, safe standard
const birthYear = 1995;
let currentVersion = "ES2023";
```

6. OUTPUT

```
Output:
(Internal memory stores these values securely in modern JS, preventing
accidental overwrites)
```

7. CODE EXPLANATION

Line 2: Implicit global variable (bad practice today).
Line 5: Using 'var' can cause scoping bugs in complex programs.
Line 8-9: Using 'const' (for values that never change) and 'let' (for values that do change) is the modern, safe way to write JavaScript.

8. REAL-WORLD USE

> Companies systematically update their old codebases from ES5 (var) to modern ES6+ (let/const) to prevent security bugs and make code easier for new developers to read.

9. COMMON MISTAKES

> **Caution:** Thinking JavaScript has stopped evolving. A new ECMAScript (ES) version is released every single year with new features!

10. PRACTICE

> **Task:** Research and write down what 'ECMAScript 6' (ES6) is and name two major features introduced in it.

JavaScript vs Java

1. CONCEPT

> **JavaScript and Java are two completely different programming languages with different purposes, despite sharing a similar name.**

2. SIMPLE EXPLANATION

The most common beginner mistake is thinking Java and JavaScript are related. They are not! The famous joke is: 'Java is to JavaScript what Car is to Carpet.' Java is a compiled, strictly-typed language built by Sun Microsystems, used heavily for enterprise software and Android apps. JavaScript is an interpreted, dynamically-typed language built by Netscape, used primarily for web interactivity. The name was purely a marketing trick to piggyback on Java's immense popularity in the 1990s.

3. SYNTAX

```
// JavaScript (Dynamic - data types are flexible)
let x = 10;
x = 'Hello'; // This is perfectly fine

// Java (Strict - data types are rigid)
int x = 10;
x = 'Hello'; // ERROR! Program crashes.
```

4. EASY EXAMPLE

Let's look at how you print a simple message to the screen in both languages to see the difference in complexity.

5. CODE

```
// --- JAVASCRIPT ---
console.log("Hello from JavaScript!");

// --- JAVA ---
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello from Java!");
    }
}
```

6. OUTPUT

```
Output:
Hello from JavaScript!
Hello from Java!
```

7. CODE EXPLANATION

JavaScript requires just one line of code to print a statement. It is lightweight and easy to start. Java requires a class definition, a main method, and strict syntax rules just to accomplish the exact same thing.

8. REAL-WORLD USE

> If you are building a banking system's backend processing engine, you might use Java. If you are building the interactive dashboard the user sees on the bank's website, you must use JavaScript.

9. COMMON MISTAKES

> **Caution:** Listing 'Java' on your resume when you actually meant 'JavaScript'. Employers view this as a massive red flag!

10. PRACTICE

> **Task: Explain to a friend in one sentence why Java and JavaScript share the same first four letters.**

Where JavaScript is used

1. CONCEPT

> **JavaScript has expanded beyond the web browser and is now used everywhere: frontend, backend, mobile, and even robotics.**

2. SIMPLE EXPLANATION

Initially, JavaScript was trapped inside the web browser. It could only manipulate webpages. However, in 2009, a technology called Node.js was invented, allowing JavaScript to run directly on computers and servers. Because of this, developers realized they could use one single language to build everything. Today, JavaScript powers interactive websites (Frontend), server databases (Backend), mobile apps for iOS and Android (React Native), desktop apps like VS Code and Slack (Electron), and even drone programming.

3. SYNTAX

```
// Frontend (Browser) specific code
window.alert('I am in a browser!');
```

```
// Backend (Node.js Server) specific code
const fs = require('fs');
fs.readFileSync('database.txt');
```

4. EASY EXAMPLE

Depending on WHERE you run JavaScript, you have access to different 'superpowers'.

5. CODE

```
// Try running this in a Web Browser Console:
console.log(window.innerWidth);
// Output: 1920 (Width of your screen)

// Try running this on a Node.js Server:
console.log(process.platform);
// Output: 'win32' or 'darwin' (Your operating system)
```

6. OUTPUT

```
Output:
(Varies depending on the environment running the code)
```

7. CODE EXPLANATION

Line 2: 'window' is an object that only exists inside a web browser. It represents the browser tab.
Line 6: 'process' is an object that only exists in Node.js. It represents the computer hardware and operating system.

8. REAL-WORLD USE

A startup company chooses to use JavaScript for their entire tech stack (MERN stack: MongoDB, Express, React, Node) so they only need to hire developers who know one language.

9. COMMON MISTAKES

Caution: Trying to use browser-specific commands (like `document.getElementById`) inside a Node.js backend server. It will crash because a server doesn't have a screen or HTML!

10. PRACTICE

Task: Name three different platforms or environments (other than a web browser) where JavaScript can run today.

JavaScript engines

1. CONCEPT

A JavaScript engine is a specialized software program that translates human-readable JS code into machine code that computers understand.

2. SIMPLE EXPLANATION

Computers are actually quite dumb; they only understand 1s and 0s (binary machine code). When you write `console.log('Hi')`, the computer has no idea what that means. A JavaScript Engine is the translator. It takes your code, parses it, compiles it into bytecode or machine code, and executes it at lightning speed. Every major web browser has its own built-in

engine. Google Chrome uses the 'V8' engine, Mozilla Firefox uses 'SpiderMonkey', and Apple Safari uses 'JavaScriptCore'. V8 is so fast and powerful that it was extracted from Chrome to create Node.js.

3. SYNTAX

Human Code -> Parser -> Abstract Syntax Tree (AST) -> Interpreter -> Machine Code -> Execution

4. EASY EXAMPLE

Engines don't just translate; they optimize. If you do the same thing 1000 times, the engine quietly compiles that specific part of the code to make it run 10x faster.

5. CODE

```
// The Engine sees this loop...
for (let i = 0; i < 50000; i++) {
    let math = (i * 2) / 4;
}
// ...and automatically optimizes the CPU instructions!
```

6. OUTPUT

```
Output:
(Code runs in milliseconds invisibly)
```

7. CODE EXPLANATION

When the engine notices this loop repeating a mathematical operation 50,000 times, a part of the engine called the 'JIT Compiler' (Just-In-Time) steps in. It optimizes the code while it is running to save memory and CPU power.

8. REAL-WORLD USE

> Google Maps runs entirely on JavaScript. The only reason zooming and panning around the globe is so smooth is because the V8 JavaScript engine optimizes the heavy mathematical calculations instantly.

9. COMMON MISTAKES

> **Caution:** Thinking you need to download and install a JavaScript engine to start coding. You already have one! It is built right into the web browser you use every day.

10. PRACTICE

> **Task: Match the Browser to its Engine: 1. Chrome, 2. Firefox. A) SpiderMonkey, B) V8.**

Running JavaScript

1. CONCEPT

> **JavaScript can be executed directly inside the web browser's built-in developer environment, requiring no special software installation.**

2. SIMPLE EXPLANATION

To write C++ or Java, you have to download compilers, configure system paths, and install heavy software. JavaScript is beautifully simple: if you have a web browser, you have a complete JavaScript development environment ready to go. You can write and run code directly in the browser's memory. This makes JavaScript the most accessible programming language in the world for beginners.

3. SYNTAX

Right-click anywhere on a webpage -> Inspect -> Click 'Console' tab -> Type code -> Press Enter.

4. EASY EXAMPLE

Let's perform a simple math calculation directly on a live website using the environment provided by the browser.

5. CODE

```
// Open your browser console right now and type:
let apples = 25;
let oranges = 15;
let totalFruit = apples + oranges;

console.log("I have " + totalFruit + " pieces of fruit.");
```

6. OUTPUT

```
Output:
I have 40 pieces of fruit.
```

7. CODE EXPLANATION

Lines 2-3: We tell the browser's temporary memory to hold two numbers.
Line 4: We calculate the sum.
Line 6: We print the result to the console. As soon as you refresh the page, this memory is wiped clean.

8. REAL-WORLD USE

Web developers use the browser console all day long to test small snippets of code quickly before adding them to their actual project files.

9. COMMON MISTAKES

Caution: Trying to save a large project in the browser console. The console is for temporary testing only. When you refresh, all code is lost!

10. PRACTICE

Task: Open your web browser, open the Console, and write a code that multiplies 50 by 20. Write down the exact steps you took.

Browser Console

1. CONCEPT

The Browser Console is a built-in diagnostic tool used by developers to print messages, view errors, and interact with the webpage.

2. SIMPLE EXPLANATION

If JavaScript is the engine of a car, the Browser Console is the dashboard display. When something goes wrong with the engine, the dashboard shows a 'Check Engine' light. Similarly, if your JavaScript has a bug, the Console will display a red error message explaining exactly what broke and on what line. We also use the console manually to print out hidden data (using `console.log`) to check if our code is working properly behind the scenes.

3. SYNTAX

```
console.log('Info');
console.warn('Warning');
console.error('Critical Error');
```

4. EASY EXAMPLE

You can use different types of console commands to organize your debugging messages by color and severity.

5. CODE

```
console.log("This is a standard informational message (usually white or
black).");
console.warn("This is a warning! Something might be wrong (usually
yellow).");
console.error("This is an error! The code has crashed (usually red).");

// You can even print tabular data!
console.table(["Apple", "Banana", "Cherry"]);
```

6. OUTPUT

```
Output:
This is a standard informational message (usually white or black).
[!] This is a warning! Something might be wrong (usually yellow).
[x] This is an error! The code has crashed (usually red).
(A neat table displaying the three fruits with index numbers 0, 1, 2)
```

7. CODE EXPLANATION

Line 1 is the most common tool a developer uses. Lines 2 and 3 are used to highlight issues. Line 6 is a hidden gem that formats lists of data into beautiful, easy-to-read tables inside the console.

8. REAL-WORLD USE

When building a login page, a developer might temporarily use `console.log(username)` when the user clicks submit, just to verify that the code successfully captured the typed text before sending it to the server.

9. COMMON MISTAKES

Caution: Leaving `console.log()` statements in your final code when publishing a website to the public. It can expose sensitive data or look unprofessional to users who inspect your site.

10. PRACTICE

Task: Write three lines of code that will output one standard message, one warning, and one error to the console.

HTML

---

4. EASY EXAMPLE

Let's see how a standard web project connects its files using the external method.

5. CODE




```
// file: main.js (in the same folder)
console.log("The HTML file successfully loaded this JavaScript file!");
```

6. OUTPUT

```
Output:
The webpage shows 'Welcome to Module 1', and the console displays 'The HTML
file successfully loaded this JavaScript file!'
```

7. CODE EXPLANATION

Notice that the `` is placed at the very end of the ``. This is crucial! The browser reads HTML from top to bottom. If we put the script at the top, the JavaScript might try to modify HTML elements that the browser hasn't even created yet.

8. REAL-WORLD USE

Modern websites link dozens of external scripts: one for Google Analytics, one for an interactive map, and one for their own custom application logic.

9. COMMON MISTAKES

Caution: Writing code inside a script tag that ALSO has a `src` attribute. E.g., ``. The browser will ignore the code inside and only load the external file!

10. PRACTICE

Task: Write the exact HTML line required to link an external JavaScript file named 'calculator.js'.

Node.js Introduction

1. CONCEPT

Node.js is a runtime environment that allows you to run JavaScript directly on a computer or server, completely independent of a web browser.

2. SIMPLE EXPLANATION

For 14 years, JavaScript was imprisoned in the browser. In 2009, Ryan Dahl took the V8 engine out of Google Chrome, added some computer-level abilities (like reading files and talking to databases), and called it Node.js. Node.js is NOT a new language; it is just a new 'house' for JavaScript to live in. This revolutionized the industry because developers could now write their backend servers in the exact same language they used for the frontend webpage.

3. SYNTAX

```
# Command Line / Terminal execution:
node filename.js
```

4. EASY EXAMPLE

To run JavaScript in Node.js, you don't use HTML. You use the computer's terminal (Command Prompt).

5. CODE

```
// Step 1: Create a file named 'server.js' on your desktop.
// Step 2: Write this code inside it and save:

console.log("Hello from the computer terminal! No browser needed.");

// Step 3: Open your terminal, navigate to desktop, and type:
// node server.js
```

6. OUTPUT

```
Output:
Hello from the computer terminal! No browser needed.
```

7. CODE EXPLANATION

When you type `node server.js`, the Node program wakes up, reads the JavaScript file, pushes it through the V8 engine, and prints the result right there in the black terminal window.

8. REAL-WORLD USE

Netflix, Uber, and PayPal redesigned their massive backend server infrastructures to run entirely on Node.js because it is exceptionally fast at handling thousands of simultaneous user requests.

9. COMMON MISTAKES

Caution: Trying to use Node.js to change the background color of a webpage. Node.js operates on the server, it has no concept of HTML, CSS, or colors!

10. PRACTICE

Task: Explain the difference between running code in the Browser Console vs running code via Node.js.

Basic Syntax

1. CONCEPT

Syntax refers to the spelling and grammar rules of a programming language. If you break them, the computer refuses to run the code.

2. SIMPLE EXPLANATION

Just like English has rules for punctuation and capital letters, JavaScript has strict rules. 1) JavaScript is case-sensitive (`Name` and `name` are entirely different). 2) We usually end instructions with a semicolon `;` (like a period at the end of an English sentence). 3) We use 'camelCase' for naming things: the first word is lowercase, and every following word is capitalized (e.g., `myFirstVariable`), making it easy to read without spaces.

3. SYNTAX

```
let variableName = value;
```

4. EASY EXAMPLE

Let's define some basic data using proper JavaScript grammar.

5. CODE

```
let firstName = "Mithila";
let studentAge = 22;
let isEnrolled = true;

// CASE SENSITIVITY TEST
let Score = 90;
let score = 50;

console.log(Score);
console.log(score);
```

6. OUTPUT

```
Output:
90
50
```

7. CODE EXPLANATION

Lines 1-3: We use camelCase (`firstName`, `studentAge`) to name our data. Line 6 and 7 define two COMPLETELY separate memory boxes, even though they mean the same thing in English. The capital 'S' makes all the difference to the computer.

8. REAL-WORLD USE

A team of 10 developers must agree on syntax rules (like always using camelCase and semicolons) so their code looks identical and blends seamlessly together.

9. COMMON MISTAKES

Caution: Using spaces in names (e.g., `let my name = 'John';`). The space confuses the JavaScript engine, resulting in an immediate crash.

10. PRACTICE

Task: Convert this phrase into proper camelCase format: 'user account password hash'.

Comments

1. CONCEPT

Comments are notes written in the code for humans to read. The JavaScript engine completely ignores them.

2. SIMPLE EXPLANATION

Code tells the computer WHAT to do, but comments tell other humans WHY you did it. As a beginner, you will often forget what your own code does a week after writing it! Comments are your lifeline. There are two types: Single-line comments (used for brief notes) and Multi-line comments (used for long explanations or temporarily disabling chunks of broken code).

3. SYNTAX

```
// Single line comment

/*
Multi-line
comment
*/
```

4. EASY EXAMPLE

Let's see how developers annotate their code in a real file.

5. CODE

```
// Calculate the final price after 10% tax
let price = 100;
let finalPrice = price * 1.10;

console.log(finalPrice); // Prints 110 to the console

/*
  The following code is temporarily disabled because
  the discount feature is not ready yet.

  let discount = 5;
  finalPrice = finalPrice - discount;
*/
```

6. OUTPUT

```
Output:
110
```

7. CODE EXPLANATION

Line 1 and Line 4 use `//` for quick notes. Lines 6-12 use `/* */` to 'comment out' a block of code. The engine skips over everything inside those symbols, meaning the discount is never applied.

8. REAL-WORLD USE

> In professional companies, developers are often required by their managers to write detailed comments explaining complex math algorithms before the code is approved.

9. COMMON MISTAKES

> **Caution:** Over-commenting. You do not need to write `// prints hello` next to `console.log('hello')`. Only comment on the *logic* or the *why*, not the obvious.

10. PRACTICE

> **Task: Write a single-line comment and a multi-line comment containing your name and today's date.**

Statements and Expressions

1. CONCEPT

> **A JavaScript program is a list of instructions. Statements are the instructions that perform actions, while Expressions are the phrases that produce values.**

2. SIMPLE EXPLANATION

Understanding this difference is a massive step in thinking like a programmer. An **Expression** is any piece of code that 'resolves' to a single value. Think of it like a math question: `5 + 5` is an expression that becomes `10`. A **Statement** is a full sentence that commands the computer to *do* something. Think of it like an order: `let x = 10;`. The statement tells the computer to create memory; it uses the expression (`10`) to fill that memory.

3. SYNTAX

```
// Expression (produces a value)
50 + 20;

// Statement (performs an action)
let total = 50 + 20;
```

4. EASY EXAMPLE

Expressions can be slotted inside statements like batteries inside a toy.

5. CODE

```
// '20' is an expression.
// '30' is an expression.
// '20 + 30' is an expression that becomes 50.

// The entire line below is a STATEMENT:
let result = 20 + 30;

// The entire line below is another STATEMENT:
console.log(result);
```

6. OUTPUT

```
Output:
50
```

7. CODE EXPLANATION

The JavaScript engine evaluates expressions first. It looks at `20 + 30`, calculates `50`, and then executes the statement `let result = 50;`. Statements form the skeleton of your program, expressions are the meat.

8. REAL-WORLD USE

When writing advanced React.js code later in your career, you will learn that certain HTML-like structures can only accept Expressions, not Statements. Knowing the difference now will save you hours of frustration later.

9. COMMON MISTAKES

Caution: Confusing the two. You cannot put a full statement inside a place that expects a single value.

10. PRACTICE

Task: Look at this code: `let age = 18 + 4;`. Identify the Statement and identify the Expression.

Strict Mode

1. CONCEPT

> **Strict Mode is a safety feature in JavaScript that throws errors for bad coding habits that the language usually ignores silently.**

2. SIMPLE EXPLANATION

Because JavaScript was created in 10 days, early versions allowed developers to write sloppy code without throwing errors. For example, you could use a variable without declaring it first. As programs got larger, this sloppiness caused massive bugs. In ES5, developers introduced 'Strict Mode'. By adding a simple text string at the very top of your file, JavaScript activates a strict set of rules, refusing to run if you write sloppy code. It is highly recommended to ALWAYS use strict mode.

3. SYNTAX

```
"use strict";
// your strict code follows...
```

4. EASY EXAMPLE

Let's see what happens when we make a typo with and without Strict Mode.

IT STUDY HUB

5. CODE

```
// WITHOUT STRICT MODE
let myName = "John";
myname = "Peter"; // Typo! Small 'n'.
// JS silently creates a brand NEW global variable called 'myname'. No error given!

// WITH STRICT MODE
"use strict";
let myAge = 20;
myage = 25; // Typo! Small 'a'.
// JS crashes and screams: "ReferenceError: myage is not defined"
```

6. OUTPUT

```
Output:
ReferenceError: myage is not defined
```

7. CODE EXPLANATION

Without strict mode, a simple typo creates hidden memory leaks that are impossible to track down. With `"use strict";` at the top of the file, the engine protects you from yourself by loudly crashing on the exact line where the typo occurred.

8. REAL-WORLD USE

> All modern JavaScript frameworks (like React, Angular, and Vue) automatically force your code into Strict Mode behind the scenes to guarantee high-quality code.

9. COMMON MISTAKES

> **Caution:** Putting `"use strict";` at the bottom of the file. It MUST be the very first line of code, otherwise, it does absolutely nothing.

10. PRACTICE

> **Task: Write the exact line of code required to enable strict mode in a new JavaScript file.**

Developer Tools

1. CONCEPT

> **Browser Developer Tools (DevTools) are a suite of diagnostic software built into the browser for web developers to test, inspect, and debug websites.**

2. SIMPLE EXPLANATION

A mechanic has a toolbox to look under the hood of a car. A web developer has DevTools. You access it by right-clicking a webpage and selecting 'Inspect' (or pressing F12). It contains multiple tabs. The **Elements** tab lets you edit HTML/CSS live. The **Console** tab lets you run JS code and view errors. The **Network** tab shows you every image and file downloading behind the scenes. The **Sources** tab lets you pause JavaScript execution line-by-line to find complex bugs (called 'Debugging').

3. SYNTAX

```
F12 OR Ctrl+Shift+I OR Right-Click -> Inspect
```

4. EASY EXAMPLE

You can use DevTools to modify any website on the internet locally on your machine.

5. CODE

```
// Try this experiment on a real website (like google.com):
// 1. Press F12 to open DevTools.
// 2. Go to the Console tab.
// 3. Paste this code and press Enter:

document.body.style.backgroundColor = "red";
document.body.innerHTML = "

I HACKED THE MAINFRAME

";
```

6. OUTPUT

```
Output:
(The entire website turns red and all content is replaced by your giant
text)
```

7. CODE EXPLANATION

This isn't actual hacking! The DevTools only modify the downloaded copy of the website on YOUR personal computer screen. As soon as you hit refresh, a fresh copy is downloaded from the server, and your changes disappear.

8. REAL-WORLD USE

> Developers use DevTools to test how their website looks on mobile phone screens without actually needing to transfer the code to a physical phone.

9. COMMON MISTAKES

Caution: Closing DevTools when a script breaks. If something isn't working on your page, opening the DevTools Console should ALWAYS be your first reflex.

10. PRACTICE

Task: Open a website, open DevTools, and find the 'Network' tab. Refresh the page and observe how many files are downloaded to make the site work.

IT STUDY HUB

Visual Architecture & Workflows

1. JAVASCRIPT HISTORY TIMELINE

Year	Event
1995	Brendan Eich creates Mocha (later JavaScript) in 10 days for Netscape.
1997	ECMA standardized the language (ECMAScript 1).
2008	Google releases V8 engine inside Chrome, skyrocketing JS speed.
2009	Ryan Dahl releases Node.js, allowing JS on servers.
2015	ES6 is released, modernizing the entire language syntax.
Present	JavaScript remains the #1 most used language on GitHub.

2. BROWSER JAVASCRIPT WORKFLOW

USER CLICKS BUTTON

↓

BROWSER CATCHES EVENT

↓

JAVASCRIPT ENGINE EXECUTES LOGIC

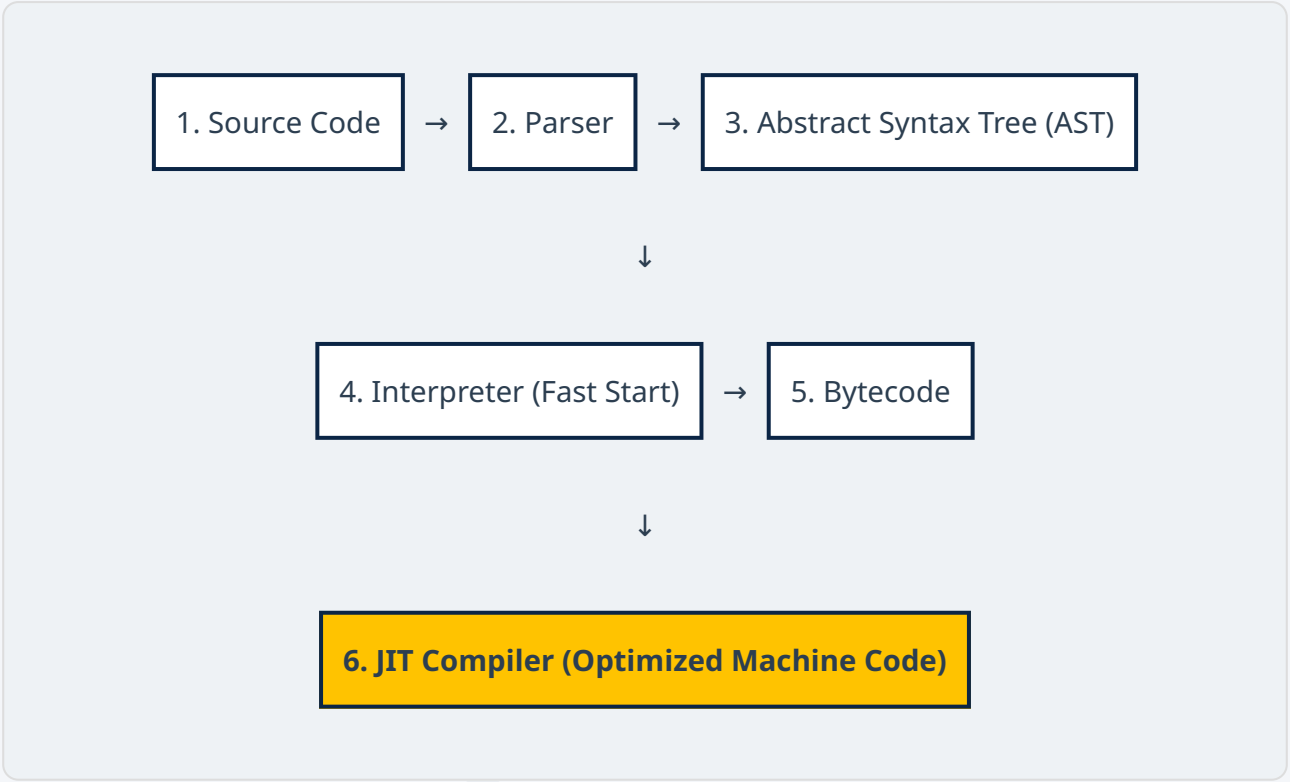
↓

DOM (HTML) IS UPDATED

↓

USER SEES NEW RESULT ON SCREEN

3. JAVASCRIPT ENGINE CONCEPT (V8)



Hands-on Project: My First Interactive Webpage

In this project, we will combine HTML (structure), CSS (style), and JavaScript (behavior) to create a webpage that changes themes when the user clicks a button. This is your first step into real-world UI engineering.

STEP 1: THE STRUCTURE (INDEX.HTML)

Create a file named `index.html`. This creates the skeleton. We define a button and link our CSS and JS files.

```
<!DOCTYPE html>
<html>
<head>
    <title>Theme Switcher</title>
    <link rel="stylesheet" href="style.css">
</head>
<body id="pageBody">
    <h1>Interactive Theme Switcher</h1>
    <button id="themeBtn">Switch to Dark Mode</button>

    <script src="app.js"></script>
</body>
</html>
```

STEP 2: THE STYLING (STYLE.CSS)

Create a file named `style.css`. We set default styles for a light theme, and prepare a specific class named `.dark-mode` that we will inject using JavaScript later.

```
body {
    font-family: Arial, sans-serif;
    background-color: white;
    color: black;
    text-align: center;
    padding-top: 100px;
    transition: 0.5s; /* Smooth color change */
}

button {
    padding: 15px 30px;
    font-size: 16px;
    cursor: pointer;
    background-color: #134074;
    color: white;
    border: none;
    border-radius: 5px;
}

/* We will apply this class using JavaScript */
.dark-mode {
    background-color: #1e1e1e;
    color: white;
}
```

STEP 3: THE LOGIC (APP.JS)

Create a file named `app.js`. We grab the HTML elements using their IDs, and attach a listener that waits for a click. When clicked, it toggles the CSS class.

```
"use strict";

// 1. Find the HTML elements
let button = document.getElementById("themeBtn");
let body = document.getElementById("pageBody");

// 2. Add an event listener to the button
button.addEventListener("click", function() {

    // 3. Toggle the dark-mode CSS class on the body
    body.classList.toggle("dark-mode");

    // 4. Update the button text depending on the theme
    if (body.classList.contains("dark-mode")) {
        button.innerHTML = "Switch to Light Mode";
    } else {
        button.innerHTML = "Switch to Dark Mode";
    }

    // 5. Log activity for debugging
    console.log("Theme switched by user.");
});
```

Code Explanation:

- `"use strict"`: We start safely to prevent bad syntax.
- `document.getElementById`: We bridge the gap between JS and HTML.
- `addEventListener`: We tell the engine to patiently wait for a specific action (a click).
- `classList.toggle`: A powerful built-in tool that adds a CSS class if it's missing, and removes it if it's present.
- `if/else`: We use basic decision-making logic to update the button's text dynamically.

Module 1 Assessment & Practice

Knowledge is solidified through repetition. Complete these sections to master the module.

15 BEGINNER EXERCISES

1. **Define JavaScript in your own words without using technical jargon.**

2. **Explain the main difference between Java and JavaScript.**

3. **Write a JavaScript statement that prints your favorite color to the console.**

4. **What year was JavaScript created, and by whom?**

5. **Name the JavaScript engine used by Google Chrome.**

6. **Explain why 'let Age = 20' and 'let age = 30' do not conflict with each other.**

7. **What does the acronym 'AST' stand for in the context of JS engines?**

8. **Write the HTML tag used to import an external JavaScript file named 'script.js'.**

9. **What is the purpose of Node.js?**

10. **Write a single-line comment and a multi-line comment in JavaScript.**

IT STUDY HUB

11. Identify the error: `let full name = 'Dithsa Dutta';`

12. What happens if you forget to include "use strict"; in modern standard scripts?

13. List three tabs found in standard Browser Developer Tools.

14. What keyboard shortcut opens the Developer Tools in most browsers?

15. Differentiate between a statement and an expression with examples.

IT STUDY HUB

20 VIVA QUESTIONS & ANSWERS

Question	Answer
What is the original name of JavaScript?	Mocha.
Is JavaScript a compiled or interpreted language?	Historically interpreted, but modern engines use Just-In-Time (JIT) compilation.
Can JavaScript run outside the web browser?	Yes, using environments like Node.js.
What is ECMAScript?	The official standard specification that JavaScript follows.
What is a JavaScript Engine?	A program that executes JavaScript code, like V8 or SpiderMonkey.
What is the difference between console.log and console.error?	log prints standard output, error prints formatted red text to highlight critical issues.
Where is the best place to put the	